claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_c5f5bca5-d2a\agent-a9322dff70f6addaa.jsonl`
- SHA-256: `7d7abd83efb9520ac60c16798cb6791bfcc3d7d736032b146845ce984bd5ec1e`
- Source modified: `2026-06-12T10:45:52+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_c5f5bca5-d2a`
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`

Transcript

1. user (2026-06-12T10:43:40.265Z)

Repo: C:\Users\avidu\Projects\sports-data-collector (Windows). Review commit fd530b5 on branch feat/prep-annotation-proxy (diff vs master: run `git diff master...feat/prep-annotation-proxy` in that repo, or read the files directly).

Changed files:
- src/core/proxy.py (NEW - probe/encode/verify/hash/manifest engine)
- src/cli/curate.py (new prep_annotation_proxy method + argparse subparser + dispatch)
- tests/test_proxy.py (NEW - 22 offline tests)
- docs/INGESTION_PIPELINE.md (step 0 + 2 FAQ entries)

Purpose (ADR-002 in docs/DECISIONS.md): re-encode a source video into a downscaled "annotation proxy" for an annotation vendor, while preserving the EXACT frame count / fps / timeline so frame N on the proxy == frame N on the original (the vendor's CVAT labels are recomputed as seconds=frame/fps by src/parsers/cvat/rally_cvat.py). Invariants: never change fps, never trim, verify parity after encode (delete proxy on failure), record provenance (source+proxy MD5) in a manifest CSV. Suite is 159 green; a live ffmpeg-8.1.1 E2E smoke passed.

A reviewer claims the following defect. ADVERSARIALLY VERIFY it by reading the actual code/files ? try to REFUTE it. Default to refuted if the scenario cannot actually occur or the behavior is acceptable/intended for this tool's scope.

CLAIM [high] Failed or interrupted encode leaves a partial proxy on disk (violates the ADR-002 'never shared' invariant) (C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:701-706)
Deletion-on-failure only happens on the PARITY branch (lines 713-719). If `run_ffmpeg` raises (disk full, codec/filter error mid-encode, odd-height failure, ffmpeg <5.1 rejecting -fps_mode after output open) or `probe_video(out)` raises on a corrupt output, the except branch prints and sys.exit(1)s WITHOUT removing `out` ? a partial `<stem>_proxy.mp4` stays in data/videos/proxies/ under exactly the name that gets shared with the vendor. The module docstring (src/core/proxy.py lines 15-17) says 'a failed proxy must be deleted, never shared'. Worse with --force: ffmpeg -y truncates a previous GOOD proxy first, so a mid-encode failure replaces a verified proxy with a broken partial. KeyboardInterrupt during the long 4K encode (not caught anywhere) leaves the partial too. Tests cannot catch this: test_prep_proxy_parity_failure_deletes_proxy covers only the parity branch, and there is no test for encode-failure cleanup because the behavior does not exist.
Proposed fix: Wrap encode?probe?verify in a try/except (and ideally finally/flag pattern covering KeyboardInterrupt and non-ProxyError exceptions) that os.remove(out) on ANY failure after ffmpeg starts; add an offline test where run_ffmpeg writes cmd[-1] then raises ProxyError and assert the file is gone.

2. user (2026-06-12T10:43:47.320Z)

fd530b5 feat(curate): prep-annotation-proxy - frame-identical vendor handoff re-encode (ADR-002)
4fb2d6f docs: ADR-002 - vendor annotation proxies generated in-repo (curate prep-annotation-

```
proxy) (#21)
a2d08f8 docs(cvat): update detect_issues padding rationale to config-driven values (#20)
 docs/INGESTION_PIPELINE.md |  29 ++++-
 src/cli/curate.py          | 147 ++++++++++++++++++++
 src/core/proxy.py          | 242 ++++++++++++++++++++++++++++++++++++
 tests/test_proxy.py        | 309 +++++++++++++++++++++++++++++++++++++++++++++
 4 files changed, 724 insertions(+), 3 deletions(-)
```

3. user (2026-06-12T10:43:48.208Z)

```
1       """Annotation-proxy generation for vendor handoff (ADR-002).
2
3       Vendors annotate in CVAT, where every rally timestamp is later recomputed from
4       *frame indices* (``seconds = frame / fps`` ? see `src/parsers/cvat/rally_cvat.py`).
5       A 4K/59.94 source is too heavy to frame-step, which is what pushed the pilot
6       vendor onto a ~30-frame sampling grid (~0.5 s) and broke the ±0.3 s boundary bar.
7       The fix is to hand the vendor a light **annotation proxy** of the *same timeline*:
8       downscaled and scrub-friendly, but identical in frame count, fps, and duration,
9       so frame N on the proxy IS frame N on the original and returned labels apply to
10      the original unchanged.
11
12      The ADR-002 proxy contract enforced here:
13      - video timestamps pass through untouched (``-fps_mode passthrough``, never
14        ``-r``): same frame count, same fps, no trim, no leading padding;
15      - the proxy is VERIFIED against the source (frame count / fps / duration) and a
16        failed proxy must be deleted, never shared ? a silently-wrong proxy poisons
17        golden labels;
18      - provenance is recorded (source + proxy MD5s) so the proxy?original mapping
19        survives the handoff (the indexer keys videos by file MD5; downstream
20        processing must keep running on the ORIGINAL file).
21      """
22      import csv
23      import hashlib
24      import os
25      import shutil
26      import subprocess
27      from dataclasses import dataclass
28      from typing import Any, Dict, List, Optional
29
30
31      class ProxyError(RuntimeError):
32          """A proxy step failed in a way that must stop the handoff (never degrade)."""
33
34
35      #: Columns of the provenance manifest (one row appended per generated proxy).
36      MANIFEST_FIELDS = [
37          "video_id", "source_path", "source_md5", "proxy_path", "proxy_md5",
38          "fps", "frame_count", "duration_s", "source_resolution", "proxy_resolution",
39          "encoder_settings", "created_at",
40      ]
41
42      # Encode defaults (see ADR-002). Height 1080 not 720 (table-tennis ball
43      # visibility); GOP 15 = a keyframe every ~0.25 s at 59.94 fps for snappy
44      # back-stepping; CRF 19 keeps the ball/shuttle legible.
45      DEFAULT_HEIGHT = 1080
46      DEFAULT_CRF = 19
47      DEFAULT_GOP = 15
48      DEFAULT_PRESET = "medium"
49
50
51      @dataclass
52      class ProbeInfo:
53          """The frame?time identity of one video stream, as ffprobe reports it."""
54          width: Optional[int] = None
55          height: Optional[int] = None
```

```
        avg_fps: Optional[float] = None     # avg_frame_rate (frames / duration)
        r_fps: Optional[float] = None       # r_frame_rate (the container's base rate)
        nb_frames: Optional[int] = None     # demuxed packet count == frame count
        duration: Optional[float] = None    # video stream duration, else container

        @property
        def resolution(self) -> str:
            if self.width and self.height:
                return f"{self.width}x{self.height}"
            return "unknown"


    def parse_rate(value: Optional[str]) -> Optional[float]:
        """Parse an ffprobe rational rate ('60000/1001', '30/1') to float fps."""
        if not value:
            return None
        try:
            if "/" in value:
                num, den = value.split("/", 1)
                num_f, den_f = float(num), float(den)
                if den_f == 0:
                    return None
                return num_f / den_f
            return float(value)
        except ValueError:
            return None


    def probe_video(path: str) -> ProbeInfo:
        """Probe the frame?time identity of `path` via ffprobe (loud on failure).

        Uses ``-count_packets`` (demux-only, no decode) so the frame count is exact
        yet cheap even for a 30-minute file. Unlike the fetch path's best-effort
        probe, a failure here RAISES: parity verification is the whole point of the
        proxy flow, so a missing ffprobe must stop it, not degrade it.
        """
        if not shutil.which("ffprobe"):
            raise ProxyError("ffprobe not found on PATH (install FFmpeg)")
        if not os.path.exists(path):
            raise ProxyError(f"video not found: {path}")
        import json as _json
        try:
            out = subprocess.run(
                ["ffprobe", "-v", "error", "-select_streams", "v:0",
                 "-count_packets",
                 "-show_entries",
                 "stream=width,height,avg_frame_rate,r_frame_rate,nb_read_packets,duration"
                 ":format=duration",
                 "-of", "json", path],
                stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
            ).stdout
            doc = _json.loads(out)
        except (subprocess.SubprocessError, OSError, ValueError) as e:
            raise ProxyError(f"ffprobe failed on {path}: {e}")

        streams = doc.get("streams") or []
        if not streams:
            raise ProxyError(f"no video stream found in {path}")
        s = streams[0]

        def _f(value) -> Optional[float]:
            try:
                return float(value)
            except (TypeError, ValueError):
                return None
```

```
        def _i(value) -> Optional[int]:
            try:
                return int(value)
            except (TypeError, ValueError):
                return None

        # Prefer the video stream's own duration; fall back to the container's
        # (audio encoder priming can stretch the container a few hundredths).
        duration = _f(s.get("duration"))
        if duration is None:
            duration = _f((doc.get("format") or {}).get("duration"))

        return ProbeInfo(
            width=_i(s.get("width")),
            height=_i(s.get("height")),
            avg_fps=parse_rate(s.get("avg_frame_rate")),
            r_fps=parse_rate(s.get("r_frame_rate")),
            nb_frames=_i(s.get("nb_read_packets")),
            duration=duration,
        )


    def is_vfr(info: ProbeInfo, rel_tol: float = 0.001) -> bool:
        """True when the stream looks variable-frame-rate (or we cannot tell).

        For CFR camera footage r_frame_rate == avg_frame_rate. A mismatch means the
        `frame / fps` time recompute is approximate ? conservative callers should
        require an explicit opt-in before proxying such a source.
        """
        if info.avg_fps is None or info.r_fps is None or info.avg_fps == 0:
            return True
        return abs(info.r_fps - info.avg_fps) / info.avg_fps > rel_tol


    def build_proxy_command(src: str, dst: str, height: int = DEFAULT_HEIGHT,
                            crf: int = DEFAULT_CRF, gop: int = DEFAULT_GOP,
                            preset: str = DEFAULT_PRESET) -> List[str]:
        """The ffmpeg invocation that downscales WITHOUT touching the timeline.

        Invariants (ADR-002): no ``-r`` / fps filter (fps must never change), no
        trim, ``-fps_mode passthrough`` (ffmpeg >= 5.1) so every source frame maps
        1:1 onto a proxy frame with its original timestamp. ``min(height, ih)``
        never upscales. Audio is kept (contact sound is the annotator's best
        boundary cue).
        """
        return [
            "ffmpeg", "-hide_banner", "-loglevel", "warning", "-stats", "-y",
            "-i", src,
            "-map", "0:v:0", "-map", "0:a?",
            "-vf", f"scale=-2:min({height}\\,ih)",
            "-c:v", "libx264", "-preset", preset, "-crf", str(crf),
            "-g", str(gop), "-pix_fmt", "yuv420p",
            "-fps_mode", "passthrough",
            "-c:a", "aac", "-b:a", "160k",
            "-movflags", "+faststart",
            dst,
        ]


    def run_ffmpeg(cmd: List[str]) -> None:
        """Run an ffmpeg command, streaming its progress to the console."""
        if not shutil.which("ffmpeg"):
            raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
        try:
```

```
            # No capture: -stats progress and any warnings go straight to the user.
            subprocess.run(cmd, check=True)
        except (subprocess.SubprocessError, OSError) as e:
            code = getattr(e, "returncode", "?")
            raise ProxyError(f"ffmpeg failed (exit {code}) ? proxy not produced")


    def verify_parity(src: ProbeInfo, proxy: ProbeInfo) -> List[str]:
        """Check the proxy carries the source's exact frame?time identity.

        Returns a list of human-readable failures (empty == parity holds). Frame
        count must match EXACTLY; fps within 0.1%; duration within a small slack
        (AAC priming can pad the container by a few hundredths of a second) that
        still catches any real trim.
        """
        errors = []

        if src.nb_frames is None or proxy.nb_frames is None:
            errors.append("frame count unavailable on source and/or proxy ? cannot prove
parity")
        elif src.nb_frames != proxy.nb_frames:
            errors.append(f"frame count changed: source {src.nb_frames} -> proxy
{proxy.nb_frames}")

        if src.avg_fps is None or proxy.avg_fps is None:
            errors.append("fps unavailable on source and/or proxy ? cannot prove parity")
        elif abs(proxy.avg_fps - src.avg_fps) / src.avg_fps > 0.001:
            errors.append(f"fps changed: source {src.avg_fps:.3f} -> proxy
{proxy.avg_fps:.3f}")

        if src.duration is None or proxy.duration is None:
            errors.append("duration unavailable on source and/or proxy ? cannot prove
parity")
        else:
            tol = max(2.0 / src.avg_fps, 0.2) if src.avg_fps else 0.2
            if abs(proxy.duration - src.duration) > tol:
                errors.append(f"duration changed: source {src.duration:.3f}s -> "
                              f"proxy {proxy.duration:.3f}s (tolerance {tol:.3f}s)")

        return errors


    def md5_file(path: str, chunk_size: int = 1024 * 1024) -> str:
        """MD5 of a file's bytes (the identity downstream tooling keys videos by)."""
        digest = hashlib.md5()
        with open(path, "rb") as f:
            for chunk in iter(lambda: f.read(chunk_size), b""):
                digest.update(chunk)
        return digest.hexdigest()


    def append_manifest_row(manifest_path: str, row: Dict[str, Any]) -> None:
        """Append one provenance row, writing the header only on first creation."""
        parent = os.path.dirname(os.path.abspath(manifest_path))
        os.makedirs(parent, exist_ok=True)
        is_new = not os.path.exists(manifest_path) or os.path.getsize(manifest_path) == 0
        with open(manifest_path, "a", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
            if is_new:
                writer.writeheader()
            writer.writerow({k: row.get(k, "") for k in MANIFEST_FIELDS})

```

4. user (2026-06-12T10:43:56.157Z)

```
646:    def prep_annotation_proxy(self, args):
1136:        "prep-annotation-proxy",
1225:    elif args.command == "prep-annotation-proxy":
1226:        cli.prep_annotation_proxy(args)
```

5. user (2026-06-12T10:44:00.019Z)

```
600                         writer.writerow(["start", "end"])
601                         for start, end in intervals:
602                             writer.writerow([start, end])
603
604                     manifest.append({
605                         "video_id": video.video_id,
606                         "sport": sport.value,
607                         "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
608                         "local_path": video.local_path,
609                         "video_url": video.video_url,
610                         "match_name": video.match_name,
611                         "fps": video.fps,
612                         "resolution": video.resolution,
613                         "license_type": video.license_type.value,
614                         "is_commercial": video.is_commercial,
615                         "status": video.status.value,
616                         "num_segments": len(intervals),
617                     })
618
619             os.makedirs(out_dir, exist_ok=True)
620             manifest_doc = {
621                 "commercial_only": commercial_only,
622                 "statuses": [s.value for s in statuses],
623                 "total_videos": len(manifest),
624                 "total_segments": sum(m["num_segments"] for m in manifest),
625                 "eval_sets": manifest,
626             }
627             manifest_path = os.path.join(out_dir, "manifest.json")
628             with open(manifest_path, "w", encoding="utf-8") as f:
629                 json.dump(manifest_doc, f, indent=2)
630
631             print(f"\n=== Exported eval/training set to '{out_dir}' ===")
632             if manifest:
633                 summary = [[m["sport"], m["video_id"], m["num_segments"],
634                             "YES" if m["is_commercial"] else "NO",
635                             os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
636                            for m in manifest]
637                 print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
638             print(f"\nVideos exported : {len(manifest)}")
639             print(f"Total segments  : {manifest_doc['total_segments']}")
640             if skipped_noncomm:
641                 print(f"Skipped (non-commercial) : {skipped_noncomm}  (pass --include-
noncommercial to include)")
642             if skipped_empty:
643                 print(f"Skipped (no annotations) : {skipped_empty}")
644             print(f"Manifest        : {manifest_path}\n")
645
646         def prep_annotation_proxy(self, args):
647             """Re-encode a video into a frame-identical annotation proxy (ADR-002).
648
649             The proxy is what the annotation vendor receives: downscaled and
650             scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
651             but with the source's EXACT frame count / fps / timeline, so frame N
652             on the proxy is frame N on the original and returned labels apply to
653             the original unchanged. Parity is verified after the encode; a proxy
654             that fails verification is deleted, never shared. Provenance (source +
```

```
            proxy MD5s) is appended to the proxy manifest ? downstream processing
            (import-local, export, the indexer) keeps running on the ORIGINAL file.
            """
            from src.core import proxy as proxylib

            src = args.video_path
            if not os.path.exists(src):
                print(f"Error: video '{src}' not found.")
                sys.exit(1)

            stem = os.path.basename(src).split('.')[0]
            # Mirror import-local's video_id derivation so the manifest row and a
            # later DB registration of the same source agree on the key.
            video_id = args.video_id or stem.lower().replace(" ", "_")
            out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
            if os.path.abspath(out) == os.path.abspath(src):
                print("Error: --out must differ from --video-path.")
                sys.exit(1)
            if os.path.exists(out) and not args.force:
                print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
                sys.exit(1)

            try:
                src_info = proxylib.probe_video(src)
            except proxylib.ProxyError as e:
                print(f"Error: {e}")
                sys.exit(1)
            if src_info.avg_fps is None:
                print(f"Error: could not determine fps of '{src}' ? the frame<->time "
                      f"contract needs it.")
                sys.exit(1)
            if proxylib.is_vfr(src_info) and not args.allow_vfr:
                print(f"Error: '{src}' looks variable-frame-rate "
                      f"(r={src_info.r_fps} vs avg={src_info.avg_fps}).")
                print("  The vendor pipeline recomputes times as frame/fps, which is only "
                      "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
                      "proceed anyway (timestamps stay identical to the original, but "
                      "frame/fps seconds are approximate for BOTH files).")
                sys.exit(1)

            os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
            cmd = proxylib.build_proxy_command(src, out, height=args.height,
                                               crf=args.crf, gop=args.gop,
                                               preset=args.preset)
            print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
                  f"fps {src_info.avg_fps:g} preserved)...")
            try:
                proxylib.run_ffmpeg(cmd)
                proxy_info = proxylib.probe_video(out)
            except proxylib.ProxyError as e:
                print(f"Error: {e}")
                sys.exit(1)

            errors = proxylib.verify_parity(src_info, proxy_info)
            if errors:
                print("Error: proxy FAILED frame<->time parity verification:")
                for err in errors:
                    print(f"  - {err}")
                try:
                    os.remove(out)
                    print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy "
                          f"poisons golden labels).")
                except OSError:
                    print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
                          f"share it with the vendor.")
```

```
                sys.exit(1)

            print("Hashing source + proxy (provenance)...")
            source_md5 = proxylib.md5_file(src)
            proxy_md5 = proxylib.md5_file(out)

            from datetime import datetime, timezone
            proxylib.append_manifest_row(args.manifest, {
                "video_id": video_id,
                "source_path": os.path.abspath(src),
                "source_md5": source_md5,
                "proxy_path": os.path.abspath(out),
                "proxy_md5": proxy_md5,
                "fps": f"{src_info.avg_fps:g}",
                "frame_count": src_info.nb_frames,
                "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None
else "",
                "source_resolution": src_info.resolution,
                "proxy_resolution": proxy_info.resolution,
                "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
                                     f"preset={args.preset} height<={args.height} "
                                     f"fps_mode=passthrough audio=aac160k"),
                "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
            })

            src_mb = os.path.getsize(src) / (1024 * 1024)
            out_mb = os.path.getsize(out) / (1024 * 1024)
            print("\n=== Annotation proxy ready ===")
            print(f"  Video ID    : {video_id}")
            print(f"  Source      : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5
{source_md5})")
            print(f"  Proxy       : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5
{proxy_md5})")
            print(f"  Parity      : VERIFIED ? {src_info.nb_frames} frames @
{src_info.avg_fps:g} fps on both")
            print(f"  Manifest    : {args.manifest}")
            print("\nNext:")
            print("  1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
                  "box of every rally on the exact contact/dead frame).")
            print("  2. On delivery, convert-cvat may use --video-path with either file "
                  "(identical fps/frame count by construction).")
            print("  3. import-local / export keep pointing at the ORIGINAL file ? the "
                  "indexer keys videos by its MD5.\n")

        def convert_cvat(self, args):
            """Convert a vendor CVAT rally export into our canonical per-rally CSV.

            Absorbs the CVAT bounding-box delivery format (rally fields carried as box
            attributes) on the ingestion side: each rally's time is recomputed from
            its frame range at the video's true fps, and content defects (overlaps,
            zero-duration, off-vocabulary endings, coarse sampling) are written to an
            issues report rather than silently "fixed". The emitted CSV imports via
            `import-local`; any blocker issue must be resolved first, since the
            importer's validator rejects overlapping / zero-duration rallies.
            """
            from src.parsers.cvat.rally_cvat import (
                convert, write_canonical_csv, render_issue_report,
            )
            try:
                result = convert(args.input, fps=args.fps,
                                 video_path=args.video_path, step=args.step)
            except (FileNotFoundError, ValueError) as e:
                print(f"Error: {e}")
                sys.exit(1)

```

```
781             if not result.records:
782                 print(f"Error: the CVAT export parsed to ZERO rallies ({args.input}). "
783                       f"Likely the wrong job was exported or the rally_number attribute was
"
784                       f"renamed (boxes without a matched rally_number are skipped).")
785                 sys.exit(1)
786 
787             write_canonical_csv(result.records, args.out)
788 
789             # Observability report next to the canonical CSV. Never raises.
790             from src.reporting import emit_delivery_report
791             src_name = os.path.splitext(os.path.basename(args.input))[0]
792             dur = (result.meta.get("stop_frame") / result.fps) if
(result.meta.get("stop_frame") and result.fps) else None
793             emit_delivery_report(
794                 out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
795                 records=result.records, issues=result.issues, fps=result.fps,
video_path=args.video_path,
796                 video_duration_s=dur, source_format="cvat", step=result.step,
797             )
798 
799             report = render_issue_report(result, os.path.basename(args.input))
```

6. user (2026-06-12T10:44:35.412Z)

```
28:def test_build_command_preserves_the_timeline():
44:def test_build_command_custom_knobs():
52:def test_parse_rate():
62:def test_is_vfr():
77:def test_verify_parity_ok_and_tolerates_audio_priming():
84:def test_verify_parity_catches_frame_count_change():
89:def test_verify_parity_catches_fps_change():
94:def test_verify_parity_catches_trim():
99:def test_verify_parity_loud_on_unknown_identity():
104:def test_md5_file(tmp_path):
110:def test_append_manifest_header_written_once(tmp_path):
137:def test_probe_video_parses_ffprobe_json(tmp_path):
151:def test_probe_video_requires_ffprobe(tmp_path):
159:def test_probe_video_missing_file():
165:def test_probe_video_no_video_stream(tmp_path):
207:def test_prep_proxy_happy_path(cli, tmp_path):
216:    with mock.patch("src.core.proxy.probe_video", side_effect=probes), \
217:         mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):
220:    assert os.path.exists(args.out)
236:def test_prep_proxy_parity_failure_deletes_proxy(cli, tmp_path):
246:    with mock.patch("src.core.proxy.probe_video", side_effect=probes), \
247:         mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):
251:    assert not os.path.exists(args.out), "a failed proxy must never be left on disk"
252:    assert not os.path.exists(args.manifest), "no provenance row for a failed proxy"
255:def test_prep_proxy_refuses_vfr_without_flag(cli, tmp_path):
264:    assert not os.path.exists(args.out)
267:def test_prep_proxy_allow_vfr_proceeds(cli, tmp_path):
279:    with mock.patch("src.core.proxy.probe_video", side_effect=[vfr, vfr_proxy]), \
280:         mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):
282:    assert os.path.exists(args.out)
285:def test_prep_proxy_refuses_existing_output(cli, tmp_path):
296:def test_prep_proxy_missing_source(cli, tmp_path):
303:def test_prep_proxy_out_equals_source_refused(cli, tmp_path):
```

7. user (2026-06-12T10:45:05.092Z)

```
src\cli\curate.py:523:                # The broadcast signal is a third-party video_url; a
downloaded copy
src\core\db.py:49:        finally:
```

8. user (2026-06-12T10:45:44.729Z)

```
Structured output provided successfully
```

9. assistant (2026-06-12T10:45:52.650Z)

```
Verification complete ? the claim is real (high severity). The encode/probe failure path in
`prep_annotation_proxy` leaves a partial proxy under the exact vendor-handoff name, violating
the ADR-002 invariant the module's own docstring and test assertions state, with the `--force`
path additionally able to destroy a previously verified proxy.
```